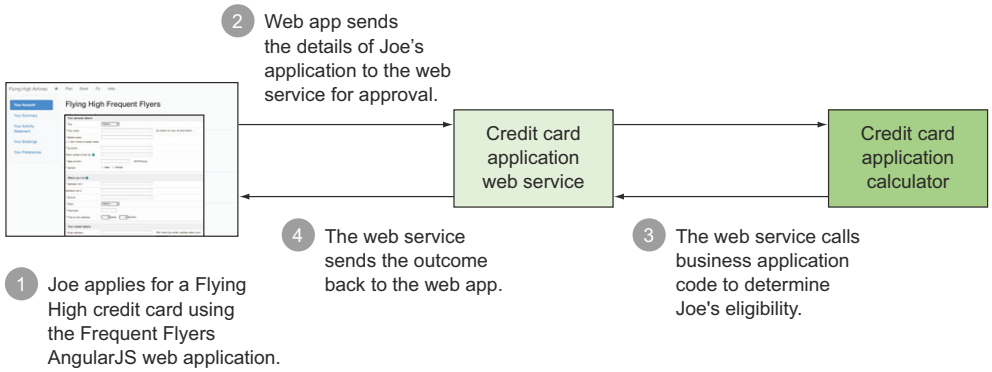


Figure 9.6 The Flying High application invokes a business process through a web service to evaluate credit card applications in real time.

You might implement this feature as illustrated in figure 9.6. Joe applies for a Frequent Flyer credit card online using an AngularJS application ❶, which submits his details to a web service to determine whether he is eligible or not ❷. The web service in turn calls a business service component that does the actual eligibility calculation ❸ and returns the result to the web page ❹ to be displayed.

In this case, you need to describe and verify the behavior of the AngularJS application, such as by using the web testing techniques discussed in chapter 8. You also need to document and verify the behavior of the web service, as we discussed in the previous section. But the core business logic around the eligibility calculation is neither in the UI nor in the web service: it's in the Credit Card Application Calculator module. There are many different scenarios that need to be verified, and it's potentially inefficient to test them all through the UI.

A more effective strategy might be to test the UI for each possible eligibility outcome (presuming they're displayed differently on the screen), and test the Credit Card Application Calculator directly to verify that the various status and income combinations produce the expected outcomes. Instead of eight UI tests, you'd have three (one for each eligibility outcome), plus eight tests written directly against the application code.

Other features may operate quite independently of the UI, and it would be difficult or impractical to test them via the UI. Batch processes, file or transaction processing, and back-end services are good examples of this type of scenario. In these cases, automating the acceptance criteria by exercising the application code directly is a viable option.

Let's look at another example. Suppose the Flying High flight-tracking system needs to process baggage registrations coming from the legacy baggage-handling system. This system provides the registrations in a well-defined text-based message format. You need to extract data from these baggage registrations and inject them into your system, routing them to different subsystems depending on whether the baggage